

Seminar Report On

Verification Methodologies for RISC-V Processors

*Submitted in partial fulfillment of the requirements for the
award of the degree of*

Bachelor of Technology

in

Computer Science and Engineering

By

Kevin Jose (U2303131)

Under the guidance of

Dr. Jomina John

**Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)**

(Parent University: APJ Abdul Kalam Technological University)

Rajagiri Valley, Kakkanad, Kochi, 682039

September 2026

CERTIFICATE

This is to certify that the seminar report entitled "**Verification Methodologies for RISC-V Processors**" is a bonafide record of the work done by **Kevin Jose (U2303131)**, submitted to Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2026–2027.

Dr. Jomina John
Associate Professor
Dept. of CSE
RSET

Dr. Jincy J. Fernandez
Associate Professor
Dept. of CSE
RSET

Dr. Mary Priya Sebastian
Associate Professor & HoD
Dept. of CSE
RSET

ACKNOWLEDGMENT

I wish to express my sincere gratitude towards Rev. Dr. Jaison Paul Mulerikkal CMI, Principal of RSET, and Dr. Mary Priya Sebastian, Associate Professor & Head of the Department of Computer Science and Engineering for providing me with the opportunity to undertake my seminar, "*Verification Methodologies for RISC-V Processors*".

I am highly indebted to my seminar coordinators, Dr. Jincy J. Fernandez, Associate Professor and Mr. Ajith S., Assistant Professor, Department of CSE, for their valuable support.

It is indeed my pleasure and a moment of satisfaction for me to express my sincere gratitude to my seminar guide, Dr. Jomina John, Associate Professor, Department of CSE, for her patience and all the priceless advice and wisdom she has shared with me.

Last but not the least, I would like to express my sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Kevin Jose

Abstract

RISC-V is an open, royalty-free instruction set architecture (ISA) that has been widely adopted across academia and industry, largely because it permits designers to add their own custom instructions. However, once a processor is fabricated a functional bug can no longer be patched, which makes pre-silicon verification critical. The official RISC-V compliance suite is handwritten and implementation-independent: it applies the same tests to every core and therefore cannot systematically target a specific pipeline depth, hazard logic, or cache configuration. Consequently, passing the compliance suite is not equivalent to being correct.

This seminar presents a study of verification methodologies for RISC-V processors through a review of four recent journal publications and an in-depth analysis of the selected base paper. The four supporting papers span the principal verification paradigms: high-level formal verification through model checking (χ RVFormal), golden-model functional verification of a vector accelerator using a UVM environment, coverage-guided RTL fuzzing (INSTILLER), and polynomial formal verification using Binary Decision Diagrams (PFV). The base paper, PATARA, proposes a self-testing framework built on the REVERSI principle, in which an operation and its inverse are applied so that a test checks itself without any golden reference model, together with a twin-based method for verifying non-invertible operations against an already-verified core.

The report discusses the methodology, system architecture, experimental setup, results, and comparative analysis of the base paper, and identifies the recurring theme across the studied literature: the two independent axes of verification, namely where test stimuli come from and how correctness is decided. The study is directly relevant to the author's project, which implements custom AI-acceleration instructions on the Shakti C-Class core, and it establishes a practical verification path for custom instructions that, by definition, have no golden reference.

List of Abbreviations

ALU	Arithmetic Logic Unit
BDD	Binary Decision Diagram
BMC	Bounded Model Checking
BSV	Bluespec SystemVerilog
CI	Continuous Integration
CSR	Control and Status Register
DUT	Design Under Test
FSM	Finite State Machine
HDL	Hardware Description Language
ISA	Instruction Set Architecture
ISS	Instruction Set Simulator
OVI	Open Vector Interface
RTL	Register Transfer Level
RVV	RISC-V Vector Extension
SIMD	Single Instruction, Multiple Data
SMT	Satisfiability Modulo Theories
SVA	SystemVerilog Assertion
UVM	Universal Verification Methodology
VACO	Vector Ant Colony Optimization
VPU	Vector Processing Unit

List of Figures

3.1	Twin-based verification framework of PATARA. Source: Gesper et al. (2026).	13
-----	--	----

List of Tables

2.1	Comparative Analysis of the Selected Papers	9
3.1	Condition Coverage by Test Configuration	14

Contents

Acknowledgment	i
Abstract	ii
List of Abbreviations	iii
List of Figures	iv
List of Tables	v
1 Introduction	1
1.1 Background	1
1.2 Overview of the Seminar Topic	1
1.3 Objectives of the Study	2
1.4 Scope of the Study	2
1.5 Organization of the Report	2
2 Literature Review	3
2.1 Overview of Existing Research	3
2.2 Review of Research Paper 1	4
2.2.1 Objective	4
2.2.2 Methodology	4
2.2.3 Key Contributions	4
2.2.4 Advantages	4
2.2.5 Limitations	5
2.3 Review of Research Paper 2	5
2.3.1 Objective	5
2.3.2 Methodology	5
2.3.3 Key Contributions	5
2.3.4 Advantages	6
2.3.5 Limitations	6
2.4 Review of Research Paper 3	6

2.4.1	Objective	6
2.4.2	Methodology	6
2.4.3	Key Contributions	7
2.4.4	Advantages	7
2.4.5	Limitations	7
2.5	Review of Research Paper 4	7
2.5.1	Objective	7
2.5.2	Methodology	7
2.5.3	Key Contributions	8
2.5.4	Advantages	8
2.5.5	Limitations	8
2.6	Comparative Analysis	8
2.7	Research Gaps Identified	9
2.8	Motivation of the Base Paper	10
3	Detailed Analysis of the Base Paper	11
3.1	Overview of the Base Paper	11
3.2	Proposed Methodology	11
3.2.1	The REVERSI Self-Testing Principle	11
3.2.2	The PATARA Tool Flow	12
3.2.3	Extensions for Pipeline Corner Cases	12
3.2.4	The Twin Method	12
3.3	System Architecture/Framework	12
3.4	Experimental Setup	13
3.5	Results and Performance Analysis	13
3.6	Comparative Analysis with Existing Methods	14
3.7	Advantages, Limitations, and Future Scope	14
4	Project Relevance and Conclusion	16
4.1	Summary of Key Findings	16
4.2	Relevance to the Proposed Project	16
4.2.1	Contribution of Seminar Concepts to the Project	17
4.3	Proposed Project Direction	17
4.4	Scope for Improvement and Future Work	17
4.5	Conclusion	18
	References	19
	Questions and Answers	20

Appendix A: Presentation**21**

Chapter 1

Introduction

1.1 Background

Modern computing increasingly depends on the ability to design processors tailored to a specific workload. The RISC-V instruction set architecture has accelerated this trend by providing an open, royalty-free, and modular ISA that anyone can implement or extend. A distinctive feature of RISC-V is its reserved encoding space for custom instructions, which allows designers to add domain-specific operations—for example, instructions that accelerate machine-learning or signal-processing workloads at the edge. This flexibility, however, shifts a significant burden onto verification: a custom processor is only useful if it can be shown to behave correctly, and once a chip is fabricated a functional error can no longer be corrected [6].

1.2 Overview of the Seminar Topic

Verification is the process of establishing that a processor implementation behaves exactly as its specification requires. For RISC-V cores this is complicated by two factors. First, the official compliance suite is handwritten and implementation-independent, so it cannot target the pipeline depth, forwarding logic, or cache organisation of any particular core. Second, most automated verification techniques rely on a *golden reference model*—a trusted implementation such as the Spike simulator—to decide whether an output is correct; but a custom instruction that a designer has invented is, by definition, absent from any such reference. This seminar surveys the principal methodologies that address these challenges: formal verification, golden-model functional verification, coverage-guided fuzzing, and self-testing.

1.3 Objectives of the Study

The main objectives of this seminar are:

- To understand the major methodologies used to verify RISC-V processor implementations.
- To analyse the working principle, strengths, and limitations of each verification approach.
- To perform a detailed study of the selected base paper on self-testing verification.
- To examine how correctness is established for a custom RISC-V core and co-processor without a golden reference model.
- To identify how these methodologies apply to the verification of custom instructions in the proposed project.

1.4 Scope of the Study

The study covers five peer-reviewed journal papers published between 2023 and 2026, each representing a distinct verification approach. It discusses the underlying principles, experimental results, advantages, and limitations of each method, with particular emphasis on how correctness is decided and where test stimuli originate. Detailed electronic-design-automation tool internals, complete formal-logic proofs, and the full micro-architecture of every evaluated core are beyond the scope of this report; the focus is on the verification methodology and its applicability to custom instruction extensions.

1.5 Organization of the Report

Chapter 1 introduces the domain, objectives, and scope. Chapter 2 reviews the four supporting papers, compares them, identifies the research gaps, and motivates the base paper. Chapter 3 analyses the base paper in detail—its methodology, architecture, experimental setup, results, and limitations. Chapter 4 summarises the key findings, relates them to the proposed project, and presents the overall conclusion.

Chapter 2

Literature Review

2.1 Overview of Existing Research

Processor verification has undergone significant development as designs have grown more complex and as open ISAs such as RISC-V have made custom cores widespread. Broadly, the field can be understood along two independent axes: *where test stimuli come from* (systematic enumeration, constrained-random generation, coverage-guided feedback, or none at all in the case of exhaustive formal methods), and *how correctness is decided* (comparison against a golden model, a self-checking property, or a mathematical proof).

Early verification relied largely on directed, hand-written tests and on the official compliance suite. While simple to apply, such tests are implementation-independent and cannot target the timing-dependent corner cases of a specific pipeline. Subsequent work introduced constrained-random stimulus generators checked against a golden reference model such as the Spike instruction-set simulator, and, more recently, coverage-guided fuzzing that uses feedback to steer test generation. In parallel, formal verification matured from verifying small blocks to proving properties of complete processors, using engines such as model checking and Binary Decision Diagrams.

The four papers reviewed in this chapter occupy different positions on the two axes and therefore frame the position of the base paper. χ RVFormal uses formal model checking against a compact reference model; the RISC-V Vector Accelerator paper uses constrained-random stimuli checked against the Spike golden model; INSTILLER uses coverage-guided fuzzing; and PFV uses BDD-based formal verification with a proven bound on verification time. Together they motivate the selection of the base paper, which uniquely avoids a golden model altogether.

2.2 Review of Research Paper 1

Paper Title: χ RVFormal: Formal Verification of RISC-V Processor Chisel Designs

Authors: S. Shen, S. Chen, Y. Liu, L. Zhang, F. Song, and Z. Wu

Year: 2026

2.2.1 Objective

The objective of this research is to make formal verification of RISC-V processors practical by replacing the large, hard-to-maintain reference model used by existing tools with a compact, high-level model that additionally covers privileged instructions and virtual memory, and by enabling the verification of pipelined and out-of-order cores.

2.2.2 Methodology

The authors develop a modular, parameterised RISC-V ISA reference model written in Chisel, a high-level hardware description language, in approximately 4,609 lines, compared with the roughly 104,911 lines of Verilog used by the existing riscv-formal tool. Because the reference model executes one instruction per step while a real processor is pipelined and may execute out of order, a synchronisation mechanism aligns the two at each instruction commit. Two strategies are provided: StateCheck, which compares the full architectural state at each commit and suits in-order cores, and ActionCheck, which compares each instruction's individual effect and scales to out-of-order cores. The resulting transition system is discharged by automated model checkers.

2.2.3 Key Contributions

- A compact Chisel reference model covering unprivileged and privileged instructions and Sv39 virtual memory.
- A synchronisation mechanism (StateCheck and ActionCheck) enabling comparison against pipelined and out-of-order designs.
- Discovery of previously unknown, designer-confirmed bugs in open-source cores.

2.2.4 Advantages

- Proves conformance to the specification for all inputs rather than for sampled tests.
- The reference model is small and maintainable.
- Scales to a complex out-of-order core (BOOM).

2.2.5 Limitations

- The reference model and synchronisation mechanism are not themselves formally verified.
- Out-of-order execution combined with virtual-memory operations is not fully supported.
- Evaluation is limited to three open-source processors, and the reference model covers only a practical subset of the ISA.

2.3 Review of Research Paper 2

Paper Title: Functional Verification of a RISC-V Vector Accelerator

Authors: V. Jiménez, M. Rodríguez, M. Domínguez, et al.

Year: 2023

2.3.1 Objective

The objective is to functionally verify Vitruvius+, an industrial RISC-V vector accelerator connected to a scalar core over the Open Vector Interface, to a level of confidence sufficient for silicon tape-out.

2.3.2 Methodology

A reusable Universal Verification Methodology (UVM) environment is built around the seven-channel Open Vector Interface, with one agent per sub-interface; only the ISSUE channel is randomised while the others react. Constrained-random vector instructions are generated using an extended version of Google’s RISC-V-DV generator, and the Spike simulator serves as the golden reference model, with results compared through a UVM scoreboard. More than fifty SystemVerilog assertions, concentrated on the memory interfaces, provide observability, and continuous-integration regressions run for approximately one year.

2.3.3 Key Contributions

- An industrial demonstration of golden-model functional verification of a vector accelerator.
- An extension of the RISC-V-DV generator to produce vector instructions.
- A detailed account of the coverage achieved and the classes of errors found.

2.3.4 Advantages

- Realistic, industry-standard methodology.
- Validated by silicon tape-out.
- High functional coverage (95.79%).
- Strong observability from interface assertions.

2.3.5 Limitations

- Verification is mainly at the interface level rather than of individual VPU sub-modules.
- Code coverage, and especially toggle coverage, remained low.
- The multi-agent UVM environment was difficult to maintain.
- The work targets the deprecated RVV 0.7.1 draft rather than the ratified specification.

2.4 Review of Research Paper 3

Paper Title: INSTILLER: Toward Efficient and Realistic RTL Fuzzing

Authors: G. Zhang, P. Wang, T. Yue, D. Liu, Y. Guo, and K. Lu

Year: 2024

2.4.1 Objective

The objective is to make coverage-guided RTL fuzzing of processors more efficient and more realistic by keeping test inputs short and by exercising interrupts and exceptions that prior fuzzers ignored.

2.4.2 Methodology

Random instruction streams are executed on both an instruction-set simulator and the RTL, and any disagreement (a “mismatch”) is flagged as a candidate bug; coverage feedback steers subsequent generation. An input-distillation technique based on ant colony optimisation (VACO) keeps test sequences short while preserving coverage: coverage points are treated as “cities” and candidate test sequences as “ants”, with pheromone reinforcing the instructions that reach coverage efficiently. Realistic multi-interrupt and exception injection with priorities, and hardware-aware seed selection and mutation, further improve effectiveness.

2.4.3 Key Contributions

- The VACO input-distillation algorithm based on ant colony optimisation.
- Realistic multi-interrupt and exception handling with priorities.
- Hardware-aware seed selection and mutation.

2.4.4 Advantages

- Achieves 29.4% higher coverage and 17.0% more mismatches than DiFuzzRTL.
- Makes test inputs 79.3% shorter, improving simulation efficiency.
- An ablation confirms that removing VACO lengthens inputs by 417%.

2.4.5 Limitations

- INSTILLER does not outperform DiFuzzRTL on every target.
- The evaluation compares mainly against a single baseline and uses four CPU cores, two of which are OpenRISC rather than RISC-V.
- VACO with relationship extraction adds overhead that offsets part of the benefit of shorter inputs.

2.5 Review of Research Paper 4

Paper Title: Polynomial Formal Verification of a RISC-V Processor

Authors: L. Weingarten, K. Datta, and R. Drechsler

Year: 2025

2.5.1 Objective

The objective is to make formal verification predictable by proving not only that a processor is correct but also a mathematical bound on the time the verification itself requires.

2.5.2 Methodology

The design is verified using Binary Decision Diagrams, a canonical representation of logic in which two equivalent circuits produce identical graphs, so equivalence checking reduces to a pointer comparison. Each instruction is isolated by pinning the control signals that select it, marking the data inputs as symbolic, and deleting nodes that collapse to constants; the remaining logic is compared against a reference BDD built

from the instruction’s mathematical definition. The control unit is verified as a finite-state-machine walk. The authors then prove that the BDD sizes—and hence the verification time—grow polynomially with the datapath width.

2.5.3 Key Contributions

- Proven polynomial complexity bounds for verification time (e.g. $O(n)$ for fetch and $O(n^2)$ for execute/decode).
- A complete correctness guarantee over all inputs.
- Verification of all 32 control signals, up from 15 in prior work.

2.5.4 Advantages

- Provides a complete correctness guarantee.
- Offers a predictable, polynomial bound on verification time.
- Addresses the unpredictable-runtime concern that limits industrial adoption of formal methods.

2.5.5 Limitations

- The evaluated core (MicroRV32) is multi-cycle and non-pipelined, with no hazards or forwarding.
- Interrupts are excluded from the scope.
- The BDD approach does not scale to multiply, divide, or floating-point units, which grow exponentially.

2.6 Comparative Analysis

The four reviewed papers, together with the base paper, demonstrate the range of verification methodologies available for RISC-V processors. χ RVFormal and PFV apply formal methods and therefore reason about all inputs, but the former bounds its proof to a fixed cycle depth while the latter applies only to a trivial, non-pipelined core. The Vector Accelerator paper and INSTILLER scale to realistic designs, but both check only sampled inputs and depend on a reference—Spike or an instruction-set simulator—that describes the standard ISA. Although each paper makes valuable contributions, none of the four supporting papers can verify a custom instruction without its reference first being extended by hand.

Table 2.1 compares the five papers across the properties most relevant to the verification of a custom RISC-V core. As with the base paper of any well-motivated

survey, the selected base paper (Paper V) is the only one that satisfies the distinguishing requirements: it needs no golden reference model, it verifies custom instructions directly, and it remains usable post-silicon.

Table 2.1: Comparative Analysis of the Selected Papers

Property	Paper I	Paper II	Paper III	Paper IV	Paper V (Base)
No golden reference model	✗	✗	✗	✗	✓
Formal proof over all inputs	✓	✗	✗	✓	✗
Detects real design bugs	✓	✓	✓	✗	✓
Scales to pipelined / OoO core	✓	✓	✓	✗	✓
Verifies custom instructions	✗	✗	✗	✗	✓
Usable post-silicon	✗	✗	✗	✗	✓

Paper I: χ RVFormal; Paper II: Vector Accelerator; Paper III: INSTILLER; Paper IV: PFV; Paper V: PATARA (base paper).

2.7 Research Gaps Identified

The literature review reveals the following gaps in existing RISC-V verification methods:

- Most methods depend on a reference—a formal model, the Spike simulator, or an instruction-set simulator—that describes only the standard ISA.
- None of the supporting methods can verify a custom, designer-defined instruction without extending its reference by hand.
- Methods that give the strongest guarantees (formal verification) either bound their proofs to a fixed depth or apply only to trivial, non-pipelined cores.
- Methods that scale to real designs (golden-model testing and fuzzing) check only sampled inputs, confirming correctness only for the tests actually run.
- Post-silicon validation is difficult for methods that require an external reference model.

2.8 Motivation of the Base Paper

The base paper, PATARA, was selected because it directly addresses the most important of these gaps. It verifies a custom RISC-V core and its co-processor *without* any golden reference model: invertible instructions check themselves through the REVERSI principle, and non-invertible operations are checked by a twin method against an already-verified core. Because it targets custom instructions that have no reference—precisely the situation of the author’s project—and because it demonstrates concretely that passing the compliance suite is not the same as being correct, it forms an ideal basis for detailed study.

Chapter Conclusion

This chapter presented a detailed review of four verification methodologies for RISC-V processors—formal model checking, golden-model functional verification, coverage-guided fuzzing, and polynomial formal verification. The comparative analysis positioned the papers along the two verification axes and identified a common limitation: every reviewed method depends on a reference for the standard ISA and therefore cannot verify a custom instruction directly. Based on these gaps, the base paper was found to be highly relevant because it establishes correctness without any golden reference model. The findings of this review provide the motivation for the detailed analysis of the base paper presented in the next chapter.

Chapter 3

Detailed Analysis of the Base Paper

3.1 Overview of the Base Paper

The base paper selected for this seminar is "A Self-Testing Framework for Verification and Validation of a RISC-V-Based System with a Co-processor" by Gesper et al., published in the International Journal of Parallel Programming (2026). The paper presents PATARA, a framework that automatically generates test programs that check themselves, so that a custom RISC-V core and an attached vector co-processor can be verified without any external golden reference model.

The central finding of the paper is that the official RISC-V compliance suite passed the authors' processor with zero detected bugs while leaving roughly a fifth of the design's logical conditions untested, whereas better, implementation-aware tests reached full coverage and revealed real bugs in the same "passing" design. This directly challenges the common assumption that passing the compliance suite is equivalent to correctness. A major contribution of the work is the twin-based method, which extends self-testing to operations that cannot be inverted by comparing them against an already-verified core.

3.2 Proposed Methodology

The methodology proposed in the base paper combines a self-checking principle, an automated generation tool, a set of extensions targeting pipeline corner cases, and a twin method for operations that cannot be inverted. The complete flow converts an ISA description into self-checking assembly tests that require no external reference.

3.2.1 The REVERSI Self-Testing Principle

Conventional verification asks whether an output equals the correct value, which forces the correct value to be computed elsewhere by a golden reference model. REVERSI

avoids this: an operation is applied and then its inverse is applied, and the result must return to the original value. Adding a number and then subtracting it must restore the starting value; if it does not, the hardware is faulty—and the correct intermediate value never needed to be known. A test therefore consists of a modification step, a restoring step, and a comparison. A key rule is that the instruction under test appears only in the modification step, so that a systematic fault cannot cancel itself out during restoration.

3.2.2 The PATARA Tool Flow

PATARA is an ISA-independent generator with three stages: instruction selection, sequence generation, and code generation. Each instruction is described in XML together with its inverse, so that the generator can be extended to a new (including custom) instruction by adding a single description rather than modifying the tool. The output is self-checking assembly that runs through the existing toolchain and simulator and needs no external reference.

3.2.3 Extensions for Pipeline Corner Cases

Plain REVERSI leaves structural blind spots, which are addressed by three extensions. Hazard-sequence generation systematically enumerates instruction combinations to exercise forwarding and stall interactions, raising condition coverage from about 81% to 97%. Operand swapping reaches the second operand’s forwarding path, which basic tests never exercise. Cache-miss generation deliberately forces misses to test miss and stall handling, closing the remaining gap to full coverage.

3.2.4 The Twin Method

REVERSI cannot verify operations that discard information, such as an absolute value or a dot product, because they cannot be inverted. For these, the same operation is computed on the hardware under test and, separately, as software on the already-verified core; the two results are then compared. Because the core is verified first by REVERSI, it can serve as the trusted reference for the co-processor, so no external simulator is required. The twin-based framework is shown in Figure 3.1.

3.3 System Architecture/Framework

As shown in Figure 3.1, a constrained-random generator produces an instruction sequence from the instruction descriptions. The sequence is executed both on the co-processor (the hardware twin) and, in parallel, as a software reference on the verified

[Insert figure here: Twin-based verification framework (Fig. 7 of the base paper)]

Figure 3.1: Twin-based verification framework of PATARA. Source: Gesper et al. (2026).

RISC-V core (the software twin). The two results are compared during a validation stage to produce the verification result. Constraints on the generator—such as valid vector-lane chaining and the avoidance of hazards on the vertical vector unit—ensure that only legal instruction sequences are generated. Because the verified RISC-V core itself provides the reference, the entire system checks itself from the bottom up, without any external golden model.

3.4 Experimental Setup

The framework was evaluated on a custom six-stage RV32IM pipeline with the stages IF, ID, EX, MEM1, MEM2, and WB. The execute stage contains separate single-cycle ALU and multi-cycle multiply and divide units, together with full forwarding logic; the design is therefore representative of a real pipelined core in which timing-dependent bugs can arise. The attached co-processor is the V²PRO vertical vector unit.

The generated self-checking tests were compiled through the standard RISC-V toolchain and executed on the register-transfer-level simulation of the core. Coverage was measured using a commercial simulator across six metrics—statement, branch, expression, condition, finite-state-machine transition, and toggle coverage—and the results were compared against those obtained by running the official RISC-V compliance suite on the same design. Condition coverage, the strictest of these metrics, was used as the primary basis for comparison.

3.5 Results and Performance Analysis

The official compliance suite executed on the core without detecting any bug, yet left condition coverage at 79.12%. PATARA reached 100% condition coverage and revealed several real bugs in the same design, all of them timing or interaction faults in control logic—for example, forwarding during a multi-cycle divide, a cache miss

during a multiply, and incorrect forwarding from a load into a JALR instruction.

A notable observation is that PATARA’s simplest configuration already exceeded the compliance suite’s coverage while using roughly 127 times fewer instructions, and that full coverage was reached only after adding the hazard-sequence and cache-miss extensions. Table 3.1 shows how condition coverage improves as each capability is added. On the co-processor, the twin method also reached 100% coverage; a tenfold increase in the number of random tests produced no improvement, and the final gap was closed only by biasing generation toward short vectors—illustrating that test *quality*, not quantity, drives coverage.

Table 3.1: Condition Coverage by Test Configuration

Configuration	Instructions	Condition coverage
Official compliance suite	889,516	79.12%
PATARA: instruction combinations	6,983	80.21%
+ operand swapping	9,722	81.31%
+ hazard sequences	1,851,852	96.70%
+ cache-miss tests	6,732,901	100.00%

Source: Prepared by the author based on the results reported in Gesper et al. (2026).

3.6 Comparative Analysis with Existing Methods

Compared with the golden-model approach of the Vector Accelerator paper, PATARA is distinguished by requiring no reference model at all: correctness is established by self-consistency for invertible instructions and by an on-chip software twin for non-invertible ones. Relative to the formal methods (χ RVFormal and PFV), PATARA is far more scalable and directly applicable to a real pipelined core, but it provides a weaker guarantee—it proves self-consistency rather than conformance to the ISA specification. The study also shows that high coverage and high bug yield are distinct properties: the golden-model campaign reported far more errors at lower coverage, whereas PATARA reports full coverage with fewer, self-reported bugs. This trade-off reinforces the central theme that no single verification method simultaneously maximises completeness, scalability, and independence from a reference.

3.7 Advantages, Limitations, and Future Scope

Advantages

- Requires no golden reference model.

- Generates tests that target the specific pipeline of the design under test.
- Achieves 100% condition coverage and finds real bugs missed by the compliance suite.
- Is extensible to custom instructions through a single XML description.
- Produces self-checking tests that remain usable post-silicon.

Limitations

- Covers RV32IM only, excluding floating point (whose rounding breaks invertibility) and the A and C extensions.
- Excludes CSRs, interrupts, and the privileged specification.
- The twin method can be defeated by an erroneous software twin and does not catch purely timing-related bugs.
- 100% coverage does not imply correctness: REVERSI proves self-consistency, not conformance to the specification.
- The bug evidence is limited to a single, self-reported design.

Future Scope

- Extending the approach to floating-point and privileged instructions.
- Combining self-testing with a formal, specification-derived reference to close the self-consistency gap.
- Confirming the discovered bugs on third-party cores.

Chapter Conclusion

This chapter presented a detailed analysis of the selected base paper, which proposes a self-testing framework for verifying a custom RISC-V core and co-processor without a golden reference model. The methodology combines the REVERSI self-checking principle, an XML-driven generation tool, three extensions for pipeline corner cases, and a twin method for non-invertible operations. The experimental analysis demonstrated that the compliance suite passed a design that in fact contained real bugs, whereas the proposed framework reached full condition coverage and exposed those bugs. Although the approach provides a weaker guarantee than formal verification, its independence from a reference model and its support for custom instructions make it directly relevant to the proposed project, which is discussed in the following chapter.

Chapter 4

Project Relevance and Conclusion

4.1 Summary of Key Findings

The literature review and the detailed analysis of the base paper provided valuable insight into how correctness is established for RISC-V processors. The reviewed papers demonstrated that verification methods differ along two axes—where test stimuli originate and how correctness is decided—and that no single method is simultaneously complete, scalable, and reference-free. Formal methods give the strongest guarantees but either bound their proofs to a fixed depth or apply only to trivial cores; golden-model testing and fuzzing scale to real designs but check only sampled inputs and depend on a reference for the standard ISA.

The base paper was shown to be the only studied method that verifies a processor without any golden model. It establishes correctness through the REVERSI self-checking principle for invertible instructions and through a twin method for non-invertible operations, and it demonstrates concretely that passing the compliance suite is not the same as being correct. These findings provide a strong technical foundation for the verification of custom instructions, which is the central concern of the proposed project.

4.2 Relevance to the Proposed Project

The concepts studied during this seminar have direct relevance to the author’s project, which implements custom AI-acceleration instructions on the Shakti C-Class core. Because these instructions are invented by the designer, no standard golden reference—such as the Spike simulator—recognises them, which is exactly the problem the base paper addresses.

An invertible custom instruction such as ADDP1 ($rd = rs1 + rs2 + 1$) maps directly onto the REVERSI principle: its inverse is a short sequence of already-verified

standard instructions, namely a subtraction followed by a decrement. A non-invertible custom instruction such as PDOT8, an eight-lane dot product, cannot be inverted because the result discards its inputs, and therefore uses the twin method. In the author’s work, PDOT8 was verified by computing the same result in hardware and in software on the verified core, producing zero mismatches over 16,384 outputs—a direct reproduction of the base paper’s methodology.

4.2.1 Contribution of Seminar Concepts to the Project

The seminar contributes three things to the project. First, it provides a concrete verification methodology for custom instructions: REVERSI self-tests for invertible operations and the twin method for non-invertible ones. Second, it establishes an awareness that the Shakti C-Class pipeline, which uses forwarding and shared decode and ALU logic that the project modifies, is exposed to precisely the timing-interaction bugs the base paper targets. Third, it provides a stronger justification for correctness than the current practice of relying on the compliance suite.

4.3 Proposed Project Direction

The proposed project is the design and verification of custom instruction-set extensions for AI acceleration on the Shakti C-Class core. Building on the knowledge gained from the seminar, each custom instruction will be verified using the methodology studied here: REVERSI self-tests for invertible operations and the twin method for non-invertible ones. The compliance-suite-based justification currently used will be replaced by systematic, self-checking tests that reach the timing-dependent corner cases the compliance suite leaves untested. Formal, specification-derived checks identified in the supporting papers are noted as a complementary direction for the standard portions of the core.

4.4 Scope for Improvement and Future Work

Although the base-paper methodology is directly applicable, several opportunities exist for extension. The verification coverage can be extended to the floating-point and compressed extensions supported by the Shakti core, which the base paper does not address. The twin method can be applied to further custom instructions as they are designed. Finally, a lightweight formal check could be introduced to close the self-consistency gap inherent in self-testing, thereby combining the scalability of self-testing with the specification conformance offered by formal verification.

4.5 Conclusion

This seminar provided a comprehensive understanding of verification methodologies for RISC-V processors. Through the literature review and the detailed analysis of the selected base paper, the principal approaches—formal model checking, golden-model functional verification, coverage-guided fuzzing, and polynomial formal verification—were studied, together with a self-testing framework that verifies a custom core and co-processor without a golden reference model.

The central lesson of the seminar is that verification is a matter of trade-offs: completeness, scalability, and independence from a reference cannot all be maximised at once. For a project built around custom instructions that have no reference, the self-testing and twin-based approach of the base paper is the most directly applicable, and it has already been used successfully to verify the project’s own custom instruction. By extending the concepts presented in the base paper, the proposed project aims to contribute towards the reliable and systematic verification of custom RISC-V instruction-set extensions for edge AI acceleration.

Bibliography

- [1] S. Gesper, F. Stuckmann, L. Wöbbekind, and G. Payá-Vayá, “A Self-Testing Framework for Verification and Validation of a RISC-V-Based System with a Co-processor,” *International Journal of Parallel Programming*, vol. 54, art. 15, 2026.
- [2] S. Shen, S. Chen, Y. Liu, L. Zhang, F. Song, and Z. Wu, “ χ RVFormal: Formal Verification of RISC-V Processor Chisel Designs,” *Journal of Systems Architecture*, vol. 175, art. 103761, 2026.
- [3] V. Jiménez et al., “Functional Verification of a RISC-V Vector Accelerator,” *IEEE Design & Test*, vol. 40, no. 3, pp. 36–44, 2023.
- [4] G. Zhang, P. Wang, T. Yue, D. Liu, Y. Guo, and K. Lu, “INSTILLER: Toward Efficient and Realistic RTL Fuzzing,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 7, pp. 2177–2190, 2024.
- [5] L. Weingarten, K. Datta, and R. Drechsler, “Polynomial Formal Verification of a RISC-V Processor,” *IEEE Transactions on Nanotechnology*, vol. 24, pp. 140–151, 2025.
- [6] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” RISC-V International, 2019.
- [7] E. Shazli and M. Tehranipoor, “REVERSI: Post-Silicon Validation via Self-Checking Randomized Test Programs,” foundational REVERSI method extended by the base paper.

Questions and Answers

1. **Does 100% coverage mean the processor is correct?**

Answer: No. It means every line, branch, and logical condition of the written code was exercised. REVERSI proves self-consistency—that a subtraction undoes an addition—rather than conformance to the RISC-V specification, so coverage is necessary but not sufficient. This is one reason formal-verification papers are also included in the study.

2. **Why is PATARA chosen as the base paper rather than the formal-verification paper?**

Answer: A base paper should be the one the project builds on. The project reproduced PATARA’s twin method on its own custom instruction. Moreover, formal verification proves conformance to the specification, but custom instructions are not in the specification, so it cannot verify the very thing the project needs.

3. **Could any of the supporting papers verify a custom instruction directly?**

Answer: Not off the shelf. Each relies on a reference—a formal model, Spike, or an instruction-set simulator—that describes only the standard ISA, so the reference would first have to be extended by hand.

4. **What did χ RVFormal actually contribute?**

Answer: Two things: a compact Chisel reference model, and, more importantly, a synchronisation mechanism (StateCheck and ActionCheck) that allows a single-cycle reference model to be compared against a pipelined or out-of-order core.

5. **How can a single-cycle reference verify an out-of-order core such as BOOM?**

Answer: An out-of-order core executes out of order but commits in program order through its reorder buffer, so the sequence of committed effects matches a single-cycle machine. The comparison is therefore performed only at instruction commit.

Appendix A: Presentation

[Insert presentation slides here using the pdfpages package.]